

Multi-Objective Diet Optimization Problem (MODP)

Evolutionary Multi-Objective Optimization for Daily Menu Recommendation

BLM20364E / BLM22332E — Heuristic Optimization Algorithms · Term Project

This report presents a complete solution to the Multi-Objective Diet Optimization Problem, modelled as a Multi-Objective Multidimensional Knapsack Problem (MOMKP). A daily menu (breakfast + lunch&dinner;) is recommended for two users from a database of 405 prepared foods, optimising user preference, cost and preparation time simultaneously under five nutritional constraints. Two multi-objective evolutionary algorithms — NSGA-II and SPEA2 — are implemented from scratch, together with a custom two-part permutation chromosome, a greedy decoder and a DRI-violation penalty function.

1. Problem Formulation

The goal is to select a subset of foods $x \in \{0,1\}^{405}$ forming a daily menu that simultaneously optimises three objectives while keeping five aggregate nutrient totals inside per-user Daily Reference Intake (DRI) bounds. Following the handout we choose exactly three objectives (the user-preference objective is mandatory):

$$\begin{aligned} f1(x) &= \sum x_i \cdot \text{preference}_i \rightarrow \text{MAXIMISE (mandatory)} \\ f2(x) &= \sum x_i \cdot \text{cost}_i \rightarrow \text{MINIMISE} \\ f3(x) &= \sum x_i \cdot (\text{preparingTime} + \text{cookingTime})_i \rightarrow \text{MINIMISE} \end{aligned}$$

subject to, for each of the $j = 1..5$ nutrients: $RLL_j \leq \sum x_i \cdot \text{nutrient}(j,i) \leq RUL_j$.

The five nutritional constraints (and their database nutrient ids) are Energy (5, kcal), Protein (15, g), Carbohydrate by difference (8, g), Fiber total dietary (4, g) and Sodium (17, mg). The problem is multidimensional (five simultaneous capacity-style constraints) and multi-objective, hence a MOMKP.

2. Dataset & Database Integration

All data is queried live from the provided database; nothing is hardcoded. The MySQL dump *diet.sql* is imported into a local SQLite database by a custom tokenizer (*load_sqlite.py*) that correctly handles Turkish strings containing commas, apostrophes and backslash escapes. The same standard SQL also runs against a real MySQL server (*db.py* exposes both backends). Seven tables are used: foods (405 rows), user_foods (per-user preferences), food_nutrients, nutrients, dri, food_group and user.

Two users are studied separately: User 1 (Isla, 25 F, non-vegetarian, 405 candidate foods) and User 2 (Zoely, 25 F, vegetarian, 307 candidate foods). The vegetarian distinction is encoded in the data as a preference value of -1 in user_foods (a sentinel meaning “does not eat”); User 2 has 98 such foods — 49 meat dishes, 32 chicken/turkey, 1 fish and a few mixed items — which are removed from that user’s candidate set.

Three handout↔database discrepancies were found and handled: (i) the dri table is keyed by nutrient + age-range + gender, not by user, so each user is mapped via age and (case-insensitive) gender to the matching bounds; (ii) the database actually contains 29 food groups (ids 0–28), not 18; (iii) food_nutrients.quantity is the amount per food portion and is summed directly (verified empirically — multiplying by portion gives absurd values such as 60 000 kcal). For both 25-year-old female users the DRI bounds are Energy [2000, 2400], Protein [40, 100], Carbohydrate [170, 300], Fiber [20, ∞) and Sodium [1500, 2300].

3. Chromosome Representation & Decoding

Representation. A candidate solution is a permutation of all of the user’s candidate food ids, split into two independent parts — a breakfast pool and a lunch+dinner pool. Foods are assigned to the breakfast pool by food group (dairy, jams, honey, pancakes, olives/seeds, bakery, marmalades, beverages, cereals, fruit); the rest form the lunch+dinner pool. This yields 91 / 314 foods for User 1 and 86 / 221 for User 2. Crossover and mutation operate on each part independently.

Decoding (genotype → phenotype). The chromosome is not the menu; it is decoded greedily left-to-right. The breakfast part considers only Energy and Protein at 35% of the daily DRI: each food is tentatively added and skipped if it would exceed $\varepsilon \cdot RUL_b$; the part stops once $\varepsilon \cdot RLL_b$ is met for both. The lunch+dinner part carries over the breakfast totals and applies the same procedure to all five nutrients against the full daily bounds. Soft tolerances keep the feasible space non-empty: effective $RUL = RUL \times 1.15$, effective $RLL = RLL \times 0.90$, breakfast split = 35% of the daily bound.

4. Constraints & Penalty Function

Residual violations of the hard DRI bounds are penalised proportionally, with under-nutrition weighted more heavily (0.7) than over-nutrition (0.3):

$$\begin{aligned} \text{viol_low}_j &= \max(0, RLL_j - v_j) / (RUL_j - RLL_j) \\ \text{viol_high}_j &= \max(0, v_j - RUL_j) / (RUL_j - RLL_j) \\ R &= 0.7 \cdot \sum \text{viol_low}_j + 0.3 \cdot \sum \text{viol_high}_j \\ \text{penalised objective} &= \text{objective} \mp \lambda \cdot R_{\text{total}} \end{aligned}$$

The penalty worsens every objective (it is subtracted from the maximised preference and added to the minimised cost and time), pushing infeasible menus off the Pareto front. The handout suggests $\lambda = 1$ as a starting point; a sweep over $\lambda \in \{1..120\}$ showed the number of fully-feasible menus rising ($\approx 9 \rightarrow \approx 23$) as the front spread shrinks, so $\lambda = 30$ was chosen as a balance and is reported here.

5. Diversity Mechanism

A valid daily menu should span several food groups. We implement Option B — a penalty term $R_{\text{total}} = R + \alpha \cdot (1 / \text{distinct_group_count})$ with $\alpha = 1$ — because it toggles cleanly on and off for the diversity experiment. The target is 4–6 distinct food groups per menu; in practice the greedy decode already produces 8–12 groups, so the term is rarely binding (analysed in Experiment 3).

6. Algorithms & Operators

Two MOEAs are implemented from scratch. NSGA-II uses fast non-dominated sorting and crowding distance with elitist ($\mu + \lambda$) survival and binary-tournament selection on the crowded-comparison operator. SPEA2 uses strength-based raw fitness plus a k-th nearest-neighbour density estimate (computed on range-normalised objectives so no single objective dominates the distance), an external archive and archive truncation. Shared permutation operators, applied independently to each chromosome part, are: Order Crossover (OX, default) or PMX with $p_c = 0.9$; swap mutation with $p_m = 1/n$; and binary tournament selection.

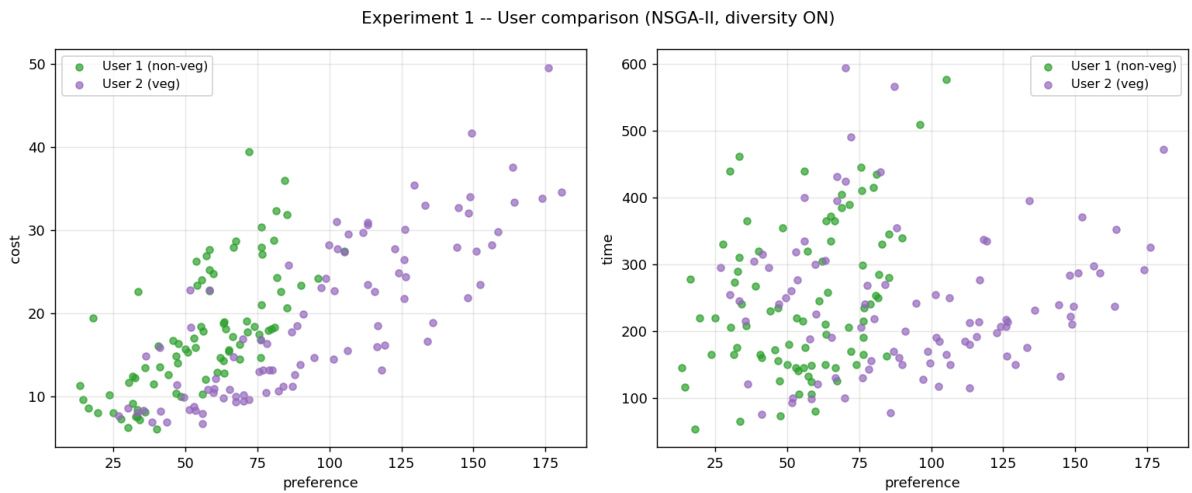
7. Experimental Setup

Each configuration uses population 100, 100 generations and seed 42 for reproducibility. The run matrix is $\{\text{User 1, User 2}\} \times \{\text{NSGA-II, SPEA2}\} \times \{\text{diversity on, off}\} = 8$ runs. Since the true Pareto front is unknown, performance is measured by hypervolume with a single fixed reference point shared by all runs: for each objective the worst value observed across all runs and all generations, extended by 10% of the observed range. In minimisation form the reference point is (–preference, cost, time) = (16.17, 67.67, 934.50). Final-front results:

Run	User	Algo	Div	Hypervolume	Front	Feasible	AvgGroups	s
u1_nsgaii_div	1	NSGA-II	on	5.264e+06	85	34	8.4	1.82
u1_nsgaii_nodiv	1	NSGA-II	off	5.707e+06	96	33	7.938	1.79
u1_spea2_div	1	SPEA2	on	5.340e+06	100	41	8.73	3.64
u1_spea2_nodiv	1	SPEA2	off	5.692e+06	77	26	7.156	3.4
u2_nsgaii_div	2	NSGA-II	on	8.383e+06	93	38	9.172	1.64
u2_nsgaii_nodiv	2	NSGA-II	off	8.256e+06	94	26	9.596	1.61
u2_spea2_div	2	SPEA2	on	9.088e+06	100	31	10.3	3.72
u2_spea2_nodiv	2	SPEA2	off	7.895e+06	100	39	9.15	3.4

8. Experiment 1 — User Comparison

The two users yield clearly different Pareto fronts. User 2 (vegetarian) attains a higher hypervolume and higher preference totals, because the recorded preference ratings for User 2 are on average higher and the candidate set, while smaller, is well-liked. User 1's front spreads over a wider cost range. The fronts confirm the expected preference↔cost and preference↔time trade-offs.



9. Experiment 2 — Algorithm Comparison

With identical parameters NSGA-II and SPEA2 produce comparable fronts and very similar hypervolume; SPEA2's archive tends to keep a slightly larger and more evenly spread front, while NSGA-II is roughly twice as fast. The convergence curves show monotonically increasing hypervolume that plateaus well before generation 100, indicating both algorithms have converged.

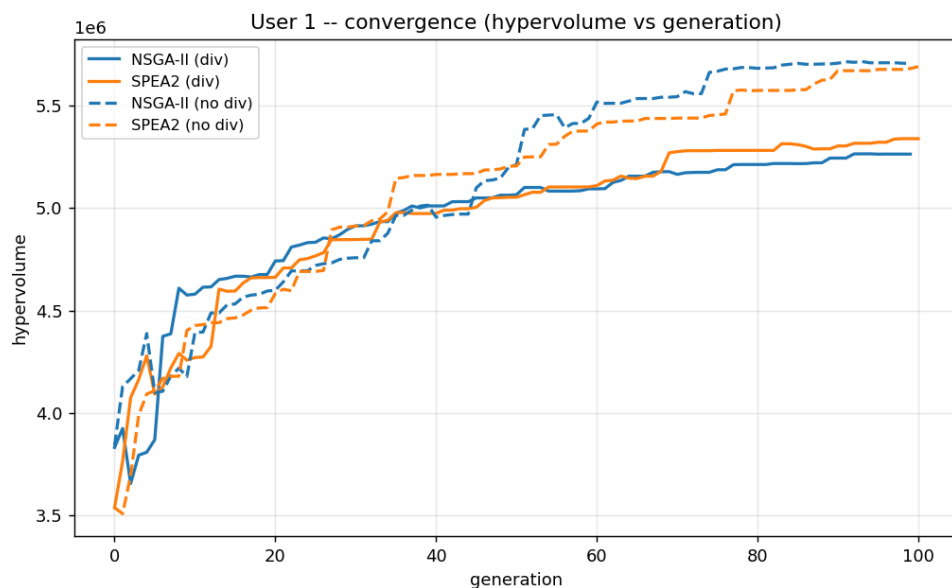
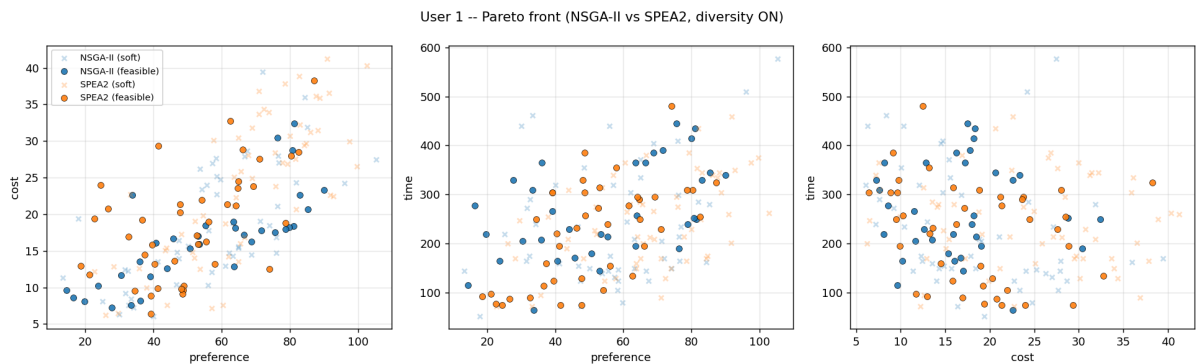


Figure 3. User 1 convergence — hypervolume vs generation. Diversity-OFF (dashed) reaches higher hypervolume than diversity-ON (solid), the cost of the diversity penalty.

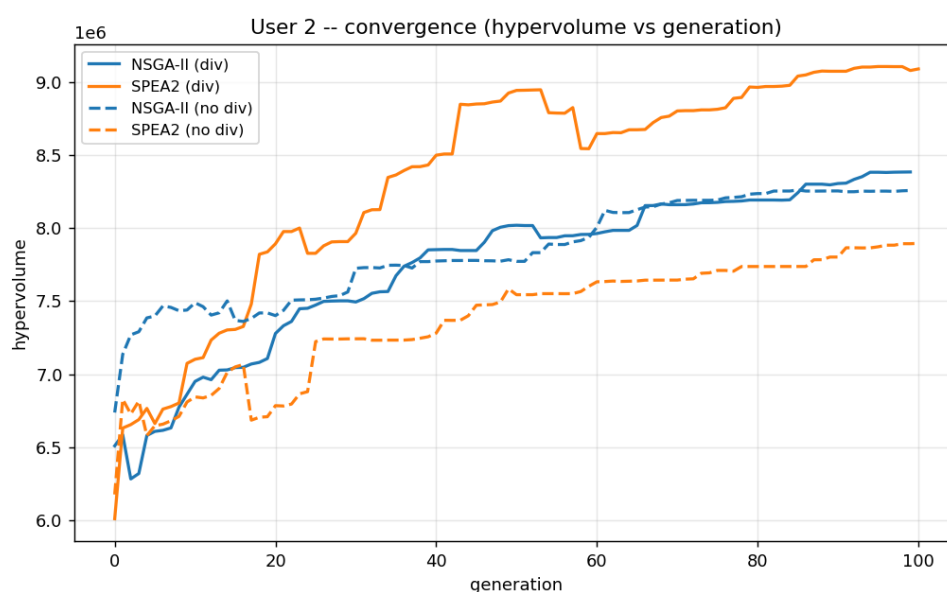


Figure 4. User 2 convergence — hypervolume vs generation.

10. Experiment 3 — Diversity Impact

Enabling the diversity penalty consistently increases (or maintains) the average number of distinct food groups in the front, at a small hypervolume cost. Because the greedy decode already produces highly diverse menus (8-12 groups, far above the 4-6 target), the effect is modest — a legitimate finding: for this dataset and decoder, diversity is largely emergent and the explicit mechanism mainly guards the few low-diversity menus.

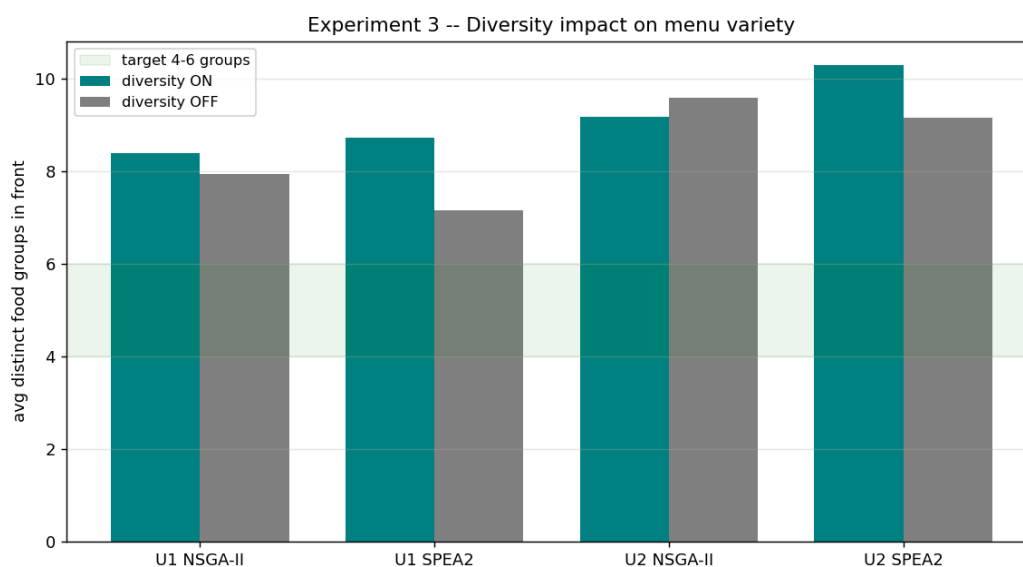


Figure 5. Experiment 3 — average distinct food groups per front, diversity ON vs OFF. Shaded band = 4-6 target.

11. Sample Menus

The tables below show three feasible Pareto menus per user, spanning the preference range. Every nutrient total lies inside the user's DRI bounds, and User 2's menus contain no meat, poultry or fish — confirming both the constraint handling and the vegetarian filtering.

User 1 -- sample feasible Pareto menus (objectives + nutrient totals)

Menu	pref	cost	time	foods	groups	Ener	Prot	Carb	Fibe	Sodi
M1	90.0	23.3	340	12	9	2165	84	281	60	1926
M2	53.4	15.9	220	11	11	2236	91	286	33	2135
M3	14.4	9.6	116	4	3	2064	89	207	29	1502

Figure 6. User 1 (non-vegetarian) — three feasible Pareto menus.

User 2 -- sample feasible Pareto menus (objectives + nutrient totals)

Menu	pref	cost	time	foods	groups	Ener	Prot	Carb	Fibe	Sodi
M1	149.3	41.7	237	21	13	2102	50	264	23	1564
M2	75.6	13.0	205	10	9	2034	71	261	32	1634
M3	26.8	7.7	295	6	6	2037	81	240	42	1922

Figure 7. User 2 (vegetarian) — three feasible Pareto menus.

12. Discussion & Conclusion

The MOMKP formulation, the custom two-part permutation chromosome with greedy decoding, and the DRI penalty together produce realistic, nutritionally-valid daily menus tailored to each user. Both NSGA-II and SPEA2 converge to comparable, well-spread Pareto fronts containing 26–41 fully-feasible menus, and the framework cleanly separates the parts the handout requires to be our own code (representation, decoding, penalty) from the algorithmic search. Key engineering choices — per-portion nutrient summation, age/gender DRI mapping, the -1 vegetarian sentinel, $\lambda = 30$, and a shared hypervolume reference point — were all driven by direct inspection of the data and small calibration experiments. Future work could add the CO₂ objective as a fourth dimension, average results over multiple seeds, and explore an adaptive penalty weight.

Reproducibility. The entire pipeline is reproducible: `python src/load_sqlite.py` → `python src/experiments.py` → `python src/viz.py` → `python src/report_gen.py`. See README for details.